OBD-II Diagnostic on STM32F429

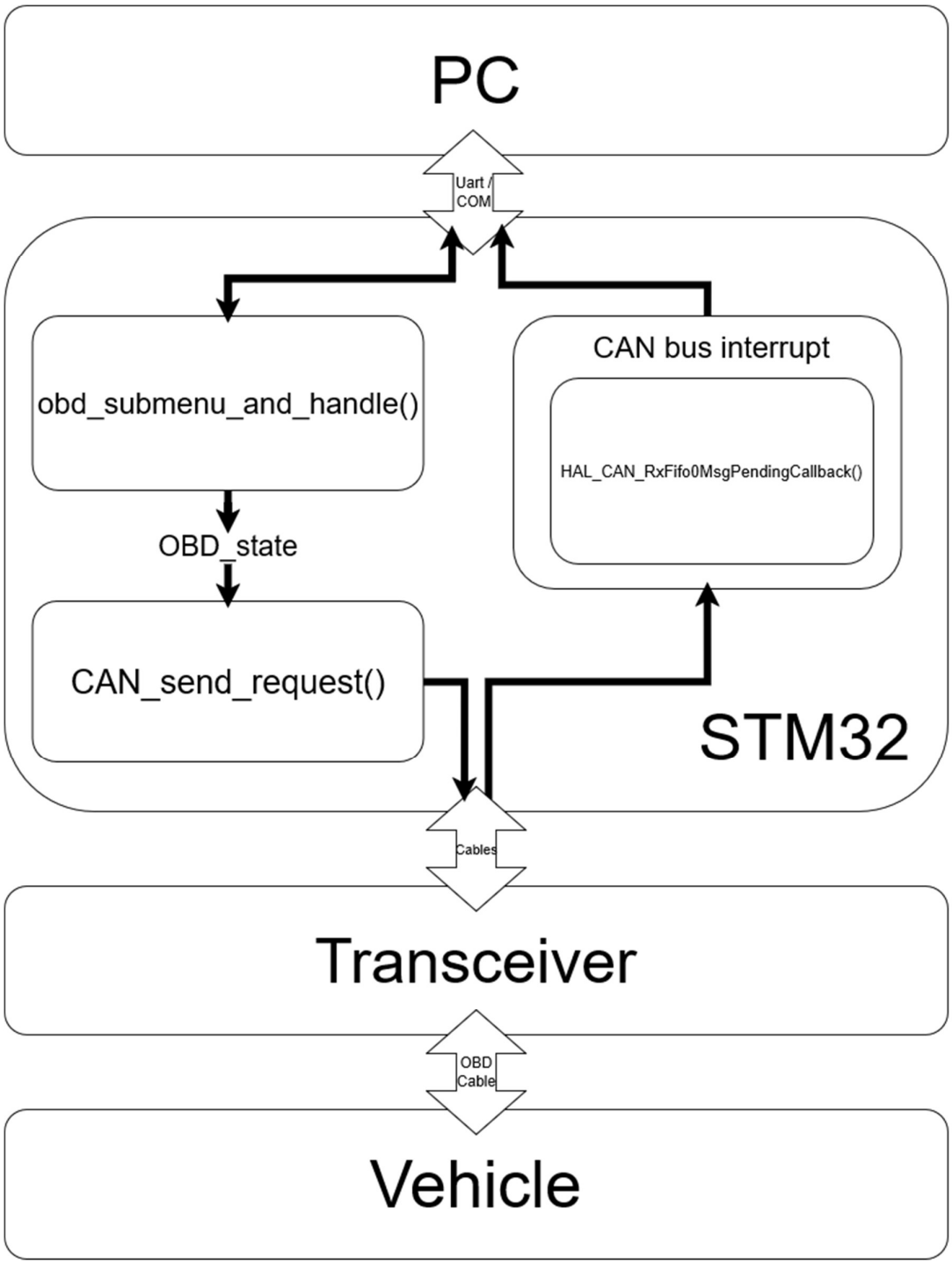
Sommario

1. Hardware 2
   1.1 System Overview 2
   1.2 MCU 4
   1.3 CAN Bus Transciver 6
   1.4 OBD-II cable 7
   1.5 Power supply 7
   1.6 UART Interface 7
2. Software 8
   2.1 Main Loop Behavior 8
   2.2 Functions 8
      2.2.1 obd_submenu_and_handle (void) 8
      2.2.2 CAN_Send_Request(void) 8
      2.2.3 HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *hcan) 9
3. CAN Bus Configuration 11
   3.1 Bitrate 11
   3.2 Filter configuration 11
4. OBD-II communication 12
   4.1 Request frame 12
   4.2 Response handling 12
5. UART menu interface 13
6. Implemented PIDs 14
7. Vehicles tested 14
8. Glossary 15

1. Hardware

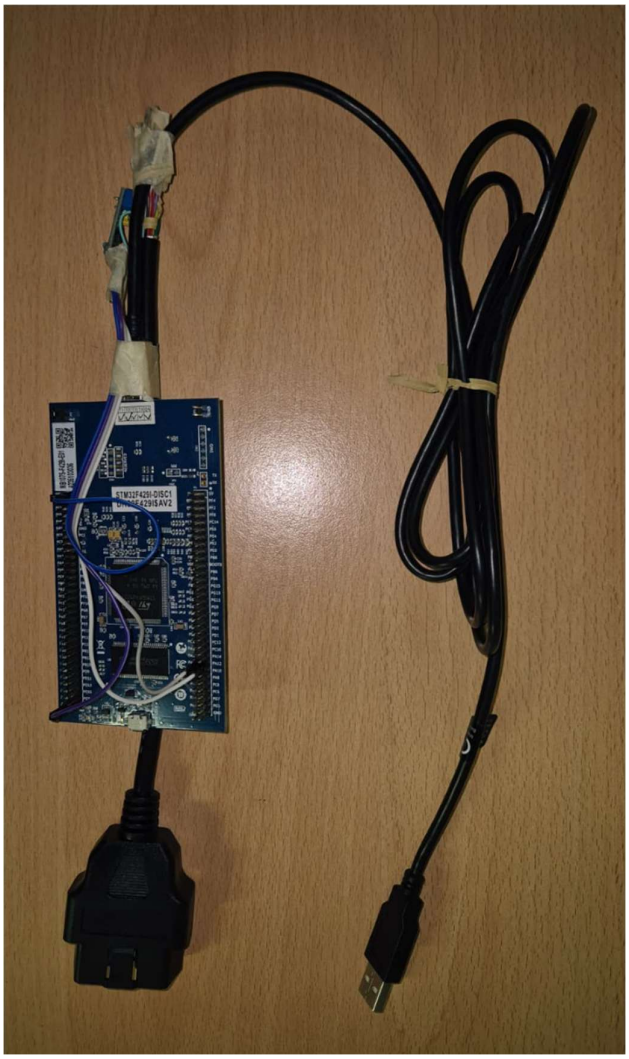
1.1 System Overview

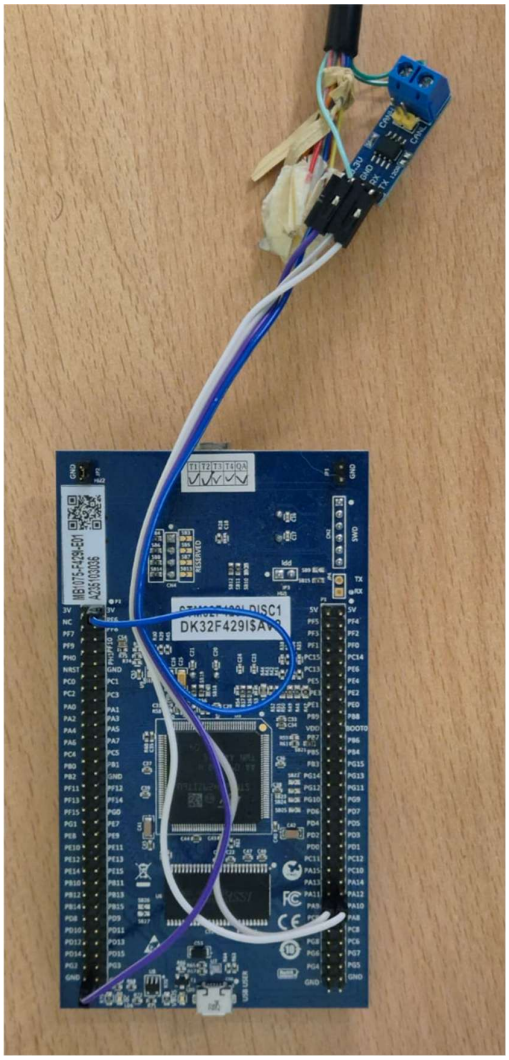
The system connects to the vehicle On-Board Diagnostic II (OBD-II) port via Controller Area Network (CAN) bus and communicates with a PC through Universal Asynchronous Receiver-Transmitter (UART).



Connections:

- OBD cable → vehicle OBD-II port
- CAN transceiver between STM32 and OBD connector
- STM32 board → PC via USB





1.2 MCU

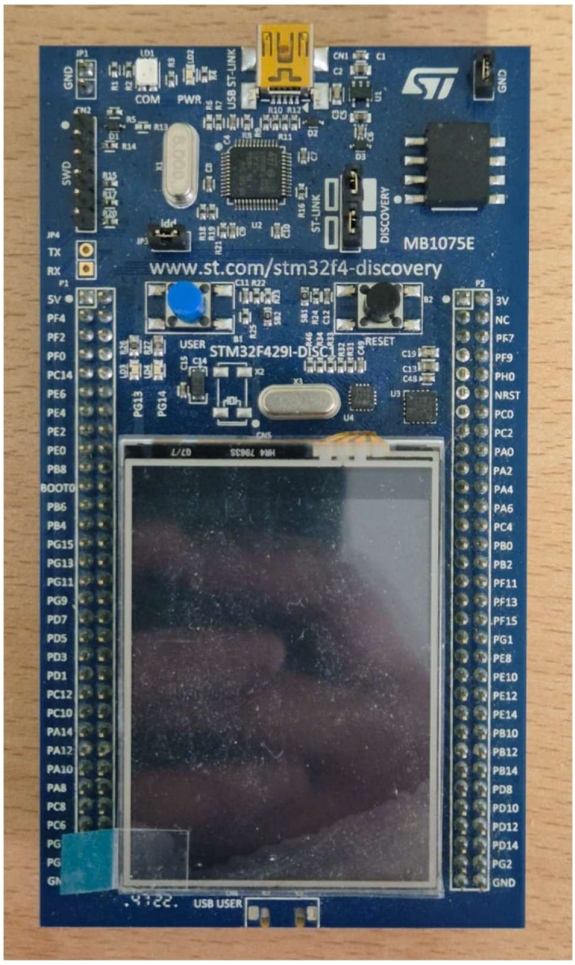
STM32 is a 32-bit microcontroller family made by STMicroelectronics.

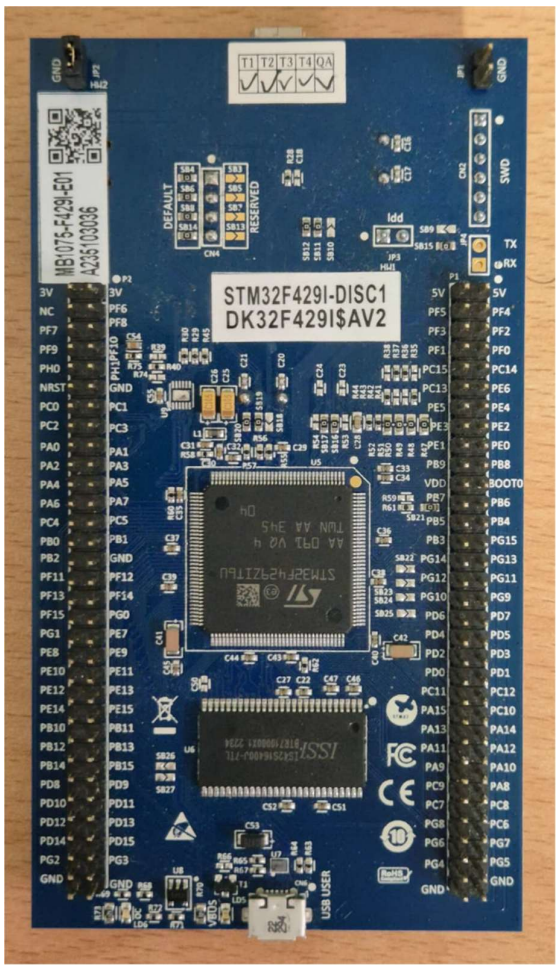
Board: STM32F429I-DISC1

MCU: STM32F429

Selected because it provides:

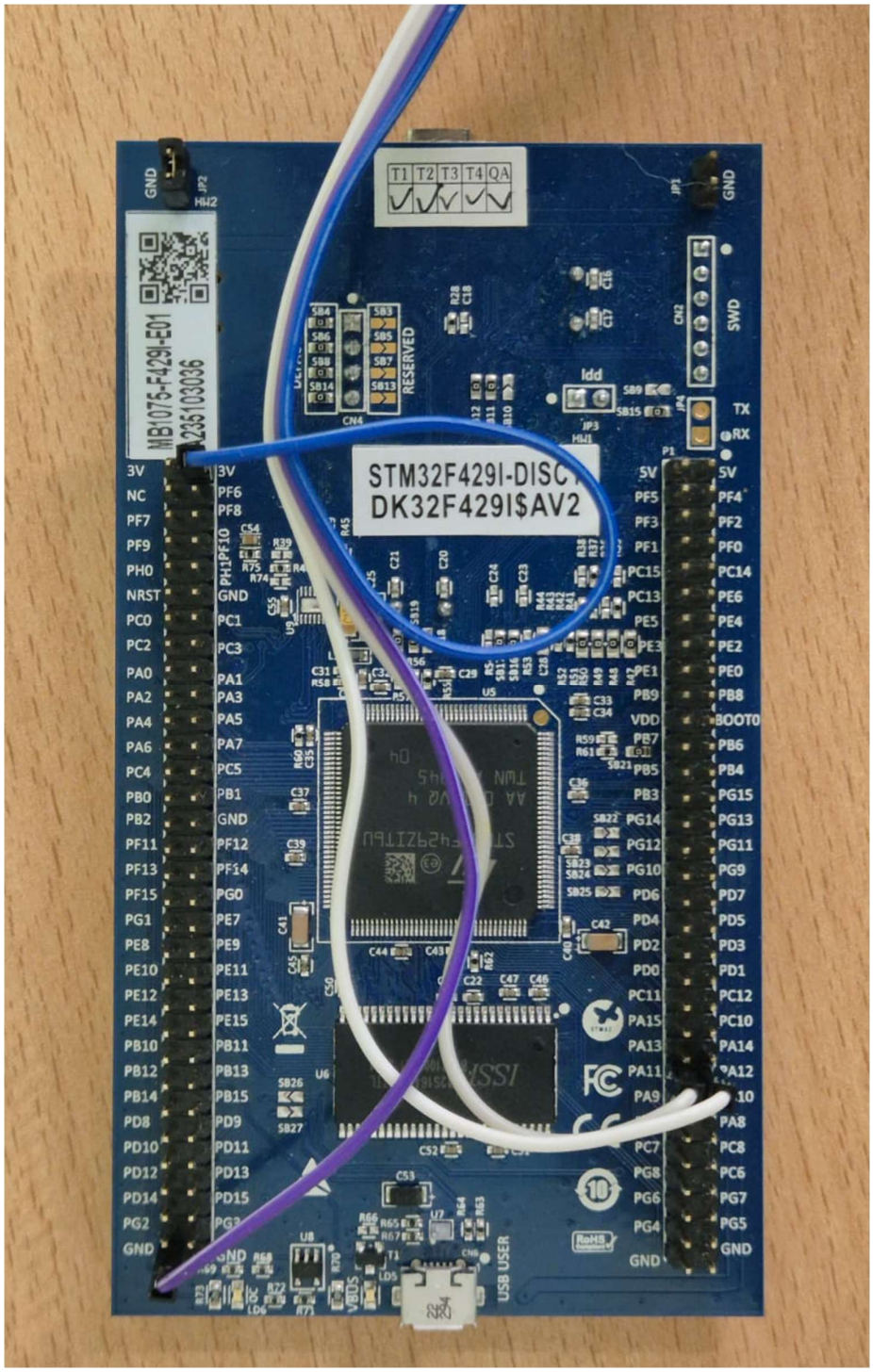
- Integrated CAN bus controller
- Universal Serial Bus (USB) interface
- Integrated ST-Link programmer
- Touchscreen (for future improvements)
- Good expansion capabilities





CAN bus pins used:

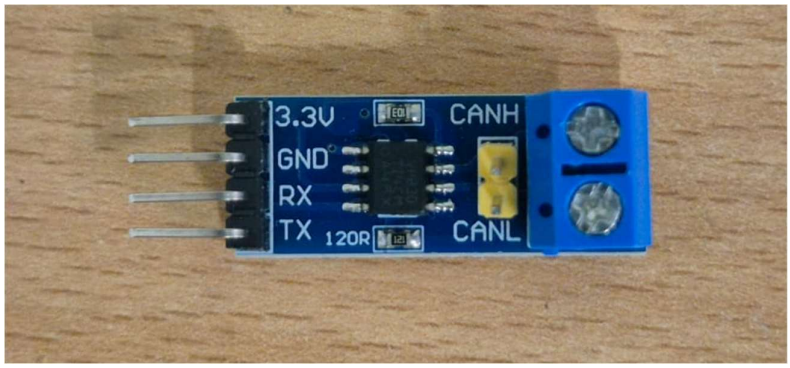
- CAN1_RX → PA11
- CAN1_TX → PA12

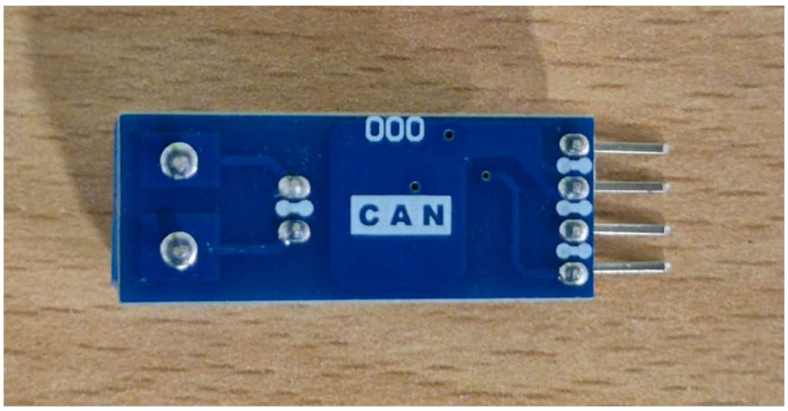


1.3 CAN Bus Transciver

Model: Texas Instruments SN65HVD230 (VP230 variant), cheap module sourced from AliExpress.

- Operates at 3.3V (compatible with STM32)
- On board 120Ω termination resistor





Connections STM32 side:

- PA11 → CAN_RX
- PA12 → CAN_TX

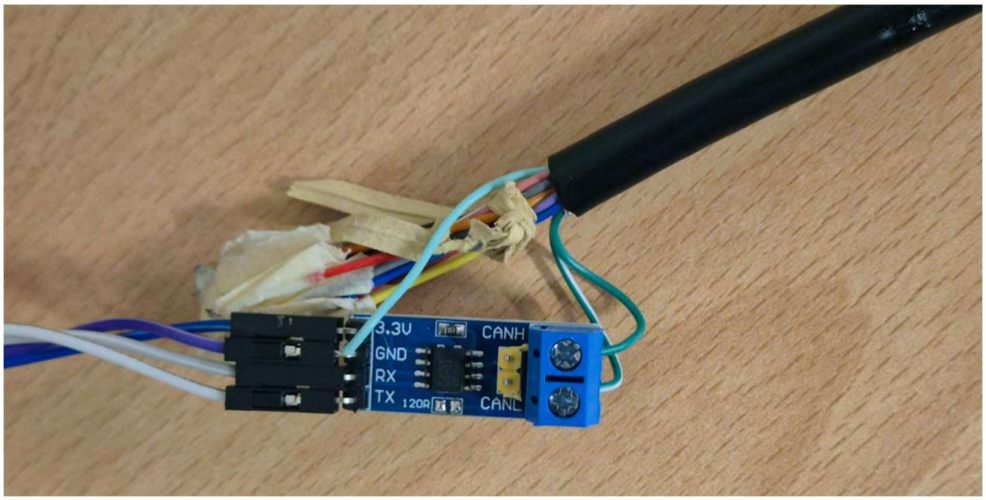
Connections OBD side:

- CANH → pin 6
- CANL → pin 14

A common ground (GND) reference was established by connecting:

- STM32 board GND
- CAN transceiver GND
- OBD-II connector Pin 5 (Vehicle Signal Ground)

This ensures proper signal reference between the vehicle CAN bus and the microcontroller system, which is required for reliable CAN communication.



1.4 OBD-II cable

Generic OBD-II cable from AliExpress, cut and stripped.

Used pins:

- Pin 6 → Vehicle CAN High
- Pin 14 → Vehicle CAN Low
- Pin 5 → Vehicle Signal Ground

1.5 Power supply

- STM32 board powered via USB (5V from PC)
- SN65HVD230 powered from STM32 3.3V rail

1.6 UART Interface

UART via integrated ST-Link Virtual Comunication (COM) Port.

Settings:

- Baudrate: 115200
- 8N1
- No flow control

Used to:

- Send commands to STM32
- Receive parsed OBD-II data
- Receive raw CAN bus frames

PC software used:

- TeraTerm
- PuTTY

2. Software

2.1 Main Loop Behavior

Inside *while(1)*:

- The system polls the selected OBD Parameter IDentification number (PID) every 500 ms via the *CAN_Send_Request()* function
- CAN bus request is transmitted
- Response is:
    - Parsed (if predefined PID)
    - Or transmitted raw over UART (generic PID mode)

To change PID selection, hardware reset button is used.

2.2 Functions

This section describes the core functions used to implement OBD-II communication over CAN bus.

2.2.1 obd_submenu_and_handle (void)

This function is called before entering the main loop. It handles user interaction via UART and selects the active OBD PID to be polled.

Description:

- Displays a submenu over UART.
- Waits for user input (single character).
- Sets a global state variable (obd_state) according to selection.
- If option "5" is selected, waits for a hexadecimal PID string (00–FF).
- Stores generic PID in generic_pid.

Behavior:

- Blocking UART reception
- Input terminates on '\r' or '\n'
- Performs minimal input validation

The menu structure is modular and can be easily extended to support additional PIDs or diagnostic services by adding new selection cases.

2.2.2 CAN_Send_Request(void)

Sends an OBD-II Mode 01 request frame over CAN bus.

Description:

- Uses standard ID 0x7DF (functional broadcast).
- Constructs an 8-byte frame:

[02] [01] [PID] [00] [00] [00] [00] [00]

- Adds message to CAN TX mailbox using:

HAL_CAN_AddTxMessage()

Notes:

- The function is called every 500 ms inside the main loop.
- PID depends on the selected state (obd_state).
- There is a similar function called "*CAN_Send_RPM_Request*" dedicated to Revolutions Per Minute (RPM) request only, used during early debugging phases to validate CAN bus communication.

2.2.3 HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *hcan)

Handles incoming CAN bus frames (interrupt-driven reception).

This function is not user-defined from scratch. It is a weak callback function already declared inside the STM32 Hardware Abstraction Layer (HAL) CAN driver (*stm32f4xx_hal_can.c*).

The HAL provides:

*__weak void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *hcan)*

The user overrides this weak definition by implementing the same function inside the application code.

Automatically triggered when:

- A new CAN bus message is received
- FIFO0 contains at least one pending message
- Interrupt *CAN_IT_RX_FIFO0_MSG_PENDING* is enabled

The function is invoked by the HAL interrupt handler *HAL_CAN_IRQHandler()*.

The callback:

1. Reads the received CAN frame using *HAL_CAN_GetRxMessage()*
2. Checks that the response is within the Engine Control Unit (ECU) response range (0x7E8 – 0x7EF)
3. Uses a global state variable (*obd_state*)
4. Selects the appropriate parsing routine through a switch statement

This state-based design makes the parsing logic modular and easily extendable. New PIDs can be added by simply:

- Extending the *obd_state* enumeration
- Adding a new case inside the switch block

The approach is conceptually similar to the modular structure used in *obd_submenu_and_handle()*, ensuring consistency in software architecture.

3. CAN Bus Configuration

3.1 Bitrate

- 500 kbps (some older car may use 250kbps)
- CAN clock: 42 MHz

Configuration:

- Prescaler = 6
- BS1 = 11
- BS2 = 2

3.2 Filter configuration

All frames are accepted.

Inside MX_CAN1_Init function:

```
CAN_FilterTypeDef filter = {0};

filter.FilterBank = 0;

filter.SlaveStartFilterBank = 14;

filter.FilterMode = CAN_FILTERMODE_IDMASK;

filter.FilterScale = CAN_FILTERSCALE_32BIT;

filter.FilterIdHigh = 0x0000;

filter.FilterIdLow = 0x0000;

filter.FilterMaskIdHigh = 0x0000;

filter.FilterMaskIdLow = 0x0000;

filter.FilterFIFOAssignment = CAN_RX_FIFO0;

filter.FilterActivation = ENABLE;


HAL_CAN_ConfigFilter(&hcan1, &filter);

HAL_CAN_Start(&hcan1);
```

This configuration allows to receive all ECU responses (0x7E8–0x7EF)

4. OBD-II communication

4.1 Request frame

Standard OBD-II request:

- ID: 0x7DF (functional broadcast)
- Data Length Code (DLC): 8 bytes

Example:

[02] [01] [PID] [00] [00] [00] [00] [00]

Where:

- 0x02 = number of additional bytes
- 0x01 = Mode 01 (Show current data)
- PID = requested parameter

4.2 Response handling

Reception handled by interrupt callback:

*void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *hcan)*

The callback:

- Reads CAN bus frame
- Verifies response ID (0x7E8–0x7EF)
- Checks Mode (0x41)
- Parses data if PID matches selected one
- Sends result via UART

5. UART menu interface

Function:

void obd_submenu_and_handle(void);

Menu options:

1 -> RPM (0x0C)

2 -> Engine Load (0x04)

3 -> Speed (0x0D)

4 -> Oil Temp (0x5C)

5 -> Inserisci PID esadecimale

Behavior:

- Single character selects predefined PID
- Option 5 allows full hexadecimal string input
- Input terminates on:
  - '\r'
  - '\n'

6. Implemented PIDs

Parsed PIDs:

- RPM (0x0C)
- Engine Load (0x04)
- Vehicle Speed (0x0D)
- Oil Temperature (0x5C)

Raw Tested PIDs:

- Coolant Temperature (0x05)
- Throttle Position (0x11)
- Intake Air Temperature (0x0F)
- Vehicle Speed (0x0D)
- Oil Temperature (0x5C)

7. Vehicles tested

- Subaru BRZ Gen 2 ZD8
- Mini R56 LCI

Note:
Oil temperature on Mini may also be available on proprietary (non-standard) PIDs.

8. Glossary

Acronym	Meaning	Page
CAN	Controller Area Network	2
COM	Communication port	7
DLC	Data Length Code	12
ECU	Engine Control Unit	9
GND	Ground	6
HAL	Hardware Abstraction Layer	9
OBD-II	On-Board Diagnostic II	2
PID	Parameter Identification number	8
RPM	Revolution Per Minute	9
STM32	32-bit microcontroller family made by STMicroelectronics	4
UART	Universal Asynchronous Receiver-Transmitter	2
USB	Universal Serial Bus	3